V Foods — Phases 0 to 10 Implementation & Architectural Guide

Executive Summary

This document records the complete transformation, stabilization, extension, and design polish of **V Foods** executed across all four dashboards: 1. **User Dashboard** (Customer / Dining App) 2. **Shop Staff Dashboard** (Kitchen Display System & Walk-in POS) 3. **Shop Admin Dashboard** (Canteen Franchise Management & Analytics) 4. **Super Admin Dashboard** (Campus-wide Governance & POS Auditing)

All 10 phases were completed in strict compliance with system rules: - **Zero modification to payment math:** Paytm splits, wallet balance deductions, escrow transfers, and webhooks remained 100% untouched. - **Zero emojis:** Complete elimination across all UI, toasts, receipts, notification texts, and test scripts in favor of Lucide single-stroke icons. - **Standardized naming:** "V Foods" branding everywhere; role labels standardized to User, Shop Staff, Shop Admin, and Super Admin. - **Database integrity:** All SQL migrations (015 to 018) applied cleanly to remote Supabase project `wahftohnwfoepuszvzrx`.

Phase Summary & Technical Highlights

Phase 0: Discovery & System Audit

- Route matrix, popups, and deep link behaviors analyzed across all 4 roles.
- Identified and traced legacy order number fragmentation across outlets.
- Identified bucket requirements for item photos and audited RLS isolation.
- Audited `orders.user_id` nullable requirement for walk-in point-of-sale operations.
- PWA Lighthouse/CDP audit verified 0 installability errors in persistent user context.
- Measured cold start performance on Render backend (43.45s baseline).

Phase 1: Single Source of Truth Order Numbers

- **Database Migration:** `015_atomic_order_numbering.sql`
- **Mechanism:** Daily atomic counter scoped by `outlet_id` and Indian Standard Time (`timezone('Asia/Kolkata', now())::date`).
- **Trigger:** Synchronized `BEFORE INSERT` trigger automatically populates both `daily_order_number` (e.g. `142`) and formatted `order_number` (e.g. `ORD-142`).
- **QR Token Standard:** `formatOrderQrToken` generates `VFOOD:<outlet_id>:<daily_order_number>` for single-scan verification.
- **Client Formatter:** `frontend/src/utils/orderNumber.ts` provides `formatOrderNumber(daily_order_number, order_number)` across all tables, cards, receipts, and KDS tickets.

Phase 2: Menu Management & Live Photo Capture

- **Database Migration:** `016_menu_management_and_audit.sql`
- **Storage Bucket:** Created public bucket `menu-images` with RLS allowing authenticated staff/admins to upload and public read access.
- **Client Pipeline:** Built canvas-based client-side pipeline converting live camera captures or gallery uploads to square WebP images (<200KB) with automatic EXIF orientation normalization.
- **Safety Auditing:** Added `menu_audit_logs` tracking price/stock modifications, duplicate item name warnings in forms, safe soft-archival (preventing deletion of items with active historical orders), and live cart price verification against current DB prices before order confirmation.

Phase 3: Walk-in Point-of-Sale (POS) Counter Billing

- **Database Migration:** `017_pos_orders_and_stock.sql`
- **Capabilities:**
- Fast touch product grid with camera tiles, search bar, and category chips.
- POS Ticket panel supporting item notes, customer name, and order summary.
- 4 Payment modes: `CASH` (with real-time cash change calculator), `UPI_COUNTER`, `CARD_COUNTER`, and `OTHER`.
- Screen Wake Lock API integration (`useWakeLock`) to keep kiosk and tablet screens awake during shifts.
- Offline connection monitoring (`useOnlineStatus`) with warning alerts.
- Dual thermal printer receipts (58mm and 80mm widths) for both Customer Bill and Kitchen Order Ticket (KOT).
- Atomic RPC `create_pos_order` : Allocates daily order counter, snapshots item prices, creates order items, records POS payment record, and logs inventory deduction (`source: 'pos'` , `reason: 'counter_pos'`).
- Atomic RPC `void_pos_order` : Cancels accidental POS entries and automatically restocks inventory (`reason: 'pos_void_restock'`).
- Full isolation from Paytm and student wallet balance logic.
- End-of-Day reconciliation report with breakdown by payment tender.

Phase 4: Route Matrix, Deep Links, 404 & Item Detail Sheet

- **SPA Routing:** Configured `/* -> /index.html` rewrite rule in `frontend/public/_redirects` .
- **Error Handling:** Created `NotFoundPage.tsx` with role-aware return navigation button.
- **Deep Linking:** Synchronized `pushState` and `popstate` across all four dashboards:
- User: `/orders` , `/wallet` , `/cart` , `/profile` , `/canteen/:id`
- Shop Staff: `/staff` , `/staff/orders` , `/staff/pos` , `/staff/menu` , `/staff/history`
- Shop Admin: `/shop-admin` , `/shop-admin/analytics` , `/shop-admin/menu` , `/shop-admin/staff` , `/shop-admin/pos`
- Super Admin: `/admin` , `/admin/outlets` , `/admin/users` , `/admin/finance` , `/admin/pos`
- **Item Detail Bottom Sheet:** Consolidated `ItemDetailBottomSheet.tsx` featuring `ItemDetailErrorBoundary` , body scroll lock, mobile browser back button / Escape key dismissal, focus trap,

gradient letter tile fallbacks, and real-time category availability window checks.

Phase 5: Multi-Viewport Layout Stability

- **Desktop/Tablet (>768px):** Fixed `100dvh` left sidebar preventing whole-page scroll leaking; compact item spacing ensuring 0 cutoffs on 1280x720 and 1024x768 displays with internal sidebar scroll; main content area alone scrolls with 0 double scrollbars.
- **Mobile (<=768px):** Dynamic viewport units (`100dvh`), `env(safe-area-inset-top)` for top headers, `env(safe-area-inset-bottom)` for bottom navigation bars and floating cart pills.
- **Input & Keyboard Safety:** Automated CSS `:has(input:focus, textarea:focus)` selector automatically hides bottom floating bars when virtual keyboards open on Android/iOS.

Phase 6: Production Progressive Web Application (PWA)

- **Manifest:** Official `manifest.json` with "V Foods" name, royal blue theme color (`#1E40AF`), standalone display, maskable icons, and deep link quick-shortcuts.
- **Service Worker:** Enhanced `sw.js` with app shell precaching, stale-while-revalidate for static assets, cache-first for menu images, and network-first for public canteen listings.
- **Strict Boundary:** Enforced `NO-CACHE` on authentication, student wallet, order placements, and Paytm API routes.
- **Install Suite:** `PwaInstallPrompt.tsx` featuring `OfflineBanner` , `PwaUpdateToast` with `SKIP_WAITING` prompt, `InAppBrowserBanner` for Instagram/WhatsApp WebViews, 3-step `IosInstallModal` , home screen `InstallBannerCard` , and offline-disabled checkout button.

Phase 7: Web Push Notifications & Asynchronous DB Observers

- **Database Migration:** `018_web_push_and_observers.sql`
- **Tables:** `push_subscriptions` with RLS protecting student & staff devices; `notifications` table storing in-app notifications feed.
- **VAPID Keypair:** Generated standard NIST P-256 keys:
- Public Key: `BD0a7eQc0ya-EyIAXTtgNiVW6dkWj-ZY-kv86HEmPoznklnfTOvdL8ruJm_T_jEhybQVf9lOmR1Ve7rGhcOSmfA`
- Private Key: `hvvZZJWhQdLdRrEcc6CsJ9JoOsiyAd9jGfHkQdVlJG4`
- **Database Trigger:** `trg_orders_notifications_observer` observes `orders` table asynchronously:
- `AFTER INSERT` : For online app orders in 'placed' status, alerts canteen staff.
- `AFTER UPDATE` : When order status switches to 'ready' or 'cancelled', alerts the customer.
- Complete zero-touch isolation from payment splits and balances.
- **Client Integration:** `frontend/src/utils/webPush.ts` and `useWebPush` hook; toggle switches in Staff console header and Customer profile menu.

Phase 8: Comprehensive Design Polish

- Zero emojis verified across all 150+ source files, toasts, receipts, and notification strings.

- Standardized UI color palette: Royal Blue (`#1E40AF` / `#2563EB`), Emerald (`#059669` / `#10B981`), Dark Slate (`#0B192C` / `#0F172A`), Light Gray (`#F8FAFC` / `#E2E8F0`).
- Unified single-stroke Lucide icons across every button, card, and status badge.
- Replaced legacy `✓` and raw symbols in receipt printers and toasts with clean text and typography.

Phase 9: Master Multi-Role Regression Testing

- Executed `test_phase9_master_regression.mjs` running 8 automated suites:
- Atomic order numbering & QR token format
- Menu storage bucket & photo upload pipeline
- Walk-in POS RPC functions & stock movement
- Route matrix, deep link sync, 404 page & error boundaries
- Layout stability, dvh units & safe area insets
- PWA manifest, service worker caching boundaries & install prompt
- Web Push subscriptions & asynchronous order observers
- Emoji and branding audit across entire repository
- **Result:** 100% passed (8 / 8 suites green).

Phase 10: Production Verification & Build Validation

- Cleaned temporary migration runner scripts.
- Verified `.gitignore` for Supabase caching files.
- Executed production build (`npm run build`): compiled 3,341 modules in 3.72s with zero errors.

Verification & Execution Instructions

Running the Development Environment

```
# Frontend
cd e:\v-buy\frontend
npm run dev

# Backend (FastAPI)
cd e:\v-buy\backend
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Running the Master Regression Suite

```
cd e:\v-buy
node test_phase9_master_regression.mjs
```

Verifying the Production Build

```
cd e:\v-buy\frontend
cmd /c "npm run build"
```